



Synchronization-part1

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Critical-Section Problem

This section demonstrates how concurrent access to shared data can produce incorrect results.

1.1 Example: Lost Updates on a Shared Counter

The shared variable is `counter`. Both threads increment this variable repeatedly without coordination.

C Code Example: `01_usleep_race_counter.c`

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 #define ITERATIONS 30
6
7 int counter = 0;
8
9 void* worker(void* arg) {
10     (void)arg;
11     for (int i = 0; i < ITERATIONS; i++) {
12         int tmp = counter;    // read shared value
13         usleep(50);           // force interleaving
14         tmp = tmp + 1;         // modify
15         counter = tmp;         // write back
16         usleep(100);
17     }
18     return NULL;
19 }
20
21 int main() {
22     pthread_t t1, t2;
23
24     pthread_create(&t1, NULL, worker, NULL);
25     pthread_create(&t2, NULL, worker, NULL);
26
27     pthread_join(t1, NULL);
28     pthread_join(t2, NULL);
29
30     printf("Final counter: %d\n", counter);
31     return 0;
32 }
```

Expected Output (?)

Final counter: 60

But is this the real output?

What Happens

Because both threads execute these steps concurrently, their operations may interleave.

Two threads may read the same value before either writes back.

Both compute the same result and write it, causing one update to be lost.

This leads to a final value smaller than expected.

Running Multiple Times

```
for i in $(seq 1 10); do ./01_usleep_race_counter; done
```

Practice Tasks

Remove both `usleep` calls and run the program multiple times. What do you observe?

Answer: After removing both `usleep` calls, the program may print 60 more often because thread interleavings become less frequent. However, the race condition still exists.

Waiting Strategies

When handling critical sections, there are two approaches:

- **Busy waiting:** the thread repeatedly checks a condition and continuously uses CPU time.
- **Blocking:** the thread sleeps while waiting and is resumed later without consuming CPU.

The next section first demonstrates busy waiting. The remaining synchronization mechanisms mainly use blocking to avoid unnecessary CPU usage.

2 Busy Waiting

In this program, one thread waits until a shared variable changes. Another thread updates the variable after a delay.

Example (`02_busy_wait_demo.c`)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 volatile int ready = 0; // shared flag
6
7 void* spinner(void* arg) {
8     (void)arg;
9     while (!ready) {
```

```
10         // busy waiting
11     }
12     printf("Spinner finished\n");
13     return NULL;
14 }
15
16 void* setter(void* arg) {
17     (void)arg;
18     sleep(2);           // simulate work
19     ready = 1;          // update condition
20     printf("Setter set ready = 1\n");
21     return NULL;
22 }
23
24 int main() {
25     pthread_t t1, t2;
26
27     pthread_create(&t1, NULL, spinner, NULL);
28     pthread_create(&t2, NULL, setter, NULL);
29
30     pthread_join(t1, NULL);
31     pthread_join(t2, NULL);
32
33     return 0;
34 }
```

What Happens

- The variable `ready` is shared between the threads.
- The `spinner` thread continuously checks the condition `while (!ready)`.
- The `setter` thread sets `ready = 1` after 2 seconds.
- When the condition becomes true, the spinner thread exits the loop and continues execution.

Measure CPU Usage

Run the original program with:

```
time ./02_busy_wait_demo
```

Example output is for example:

```
Spinner finished
Setter set ready = 1

real    0m2.006s
user    0m2.001s
sys     0m0.005s
```

Meaning of the Time Values

- **real**: total elapsed (wall-clock) time from start to finish.
- **user**: CPU time spent executing user-level code.
- **sys**: CPU time spent in the operating system (kernel).

Now modify the loop to:

```
1 while (!ready) {  
2     usleep(1000);  
3 }
```

Recompile and run again:

```
time ./02_busy_wait_demo_with_usleep
```

Example output after adding `usleep` is for example:

```
Setter set ready = 1  
Spinner finished  
  
real    0m2.007s  
user    0m0.004s  
sys     0m0.016s
```

The **real** time remains about 2 seconds because the program still waits for the same event.

Without **usleep**, the **user** time is high, meaning the CPU is continuously used.

With **usleep**, the **user** time becomes very small, showing that the thread is no longer busy waiting.

3 Mutex Locks

The previous counter example (01_race_counter.c) showed incorrect results because multiple threads updated shared data at the same time. To solve this, we need **mutual exclusion**: only one thread should access the critical section at a time. A **mutex** enforces this by allowing only one thread to hold the lock.

3.1 Basic POSIX Usage

```
1 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
2  
3 pthread_mutex_lock(&lock);  
4 /* critical section */  
5 pthread_mutex_unlock(&lock);
```

`pthread_mutex_lock` blocks if another thread holds the lock.

`pthread_mutex_unlock` releases the lock so another thread can proceed.

If one thread holds the lock, other threads are blocked until it is released.

Fixing a Race Condition (03_mutex_counter.c)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 #define ITERATIONS 30
6
7 int counter = 0;
8 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
9
10 void* worker(void* arg) {
11     for (int i = 0; i < ITERATIONS; i++) {
12         pthread_mutex_lock(&lock);
13
14         int tmp = counter;    // read shared value
15         usleep(50);          // same delay as race version
16         tmp = tmp + 1;        // modify
17         counter = tmp;        // write back
18         usleep(100);
19
20         pthread_mutex_unlock(&lock);
21     }
22     return NULL;
23 }
24
25 int main() {
26     pthread_t t1, t2;
27
28     pthread_create(&t1, NULL, worker, NULL);
29     pthread_create(&t2, NULL, worker, NULL);
30
31     pthread_join(t1, NULL);
32     pthread_join(t2, NULL);
33
34     printf("Final counter: %d\n", counter);
35     return 0;
36 }
```

Program Output

Final counter: 60

The mutex ensures that only one thread updates `counter` at a time.

This prevents lost updates and eliminates the race condition.

3.2 Correct Usage Patterns

- Always release the lock on all execution paths.
- Keep the critical section as short as possible.
- Avoid acquiring multiple locks without a fixed order.

3.3 Nonblocking Attempt

```
1 if (pthread_mutex_trylock(&lock) == 0) {
2     /* critical section */
3     pthread_mutex_unlock(&lock);
4 } else {
5     /* lock not available */
6 }
```

`pthread_mutex_trylock(&lock)` does not block and allows a thread to skip work instead of waiting.

- If the mutex (`lock`) is free, it acquires it immediately and returns 0, which allows entering the critical section.
- If the mutex is already locked, it returns immediately with an error (typically `EBUSY`), in which case it executes the else part and will not wait for the lock. It might skip work, retry later, or do alternative work.

4 Semaphores

In many situations, multiple threads need access to a limited number of resources.

For example, if only 3 resources are available, at most 3 threads should use them at the same time.

A **semaphore** is a synchronization primitive that controls how many threads can proceed concurrently.

4.1 Basic POSIX Usage

```
1 sem_t sem;
2 sem_init(&sem, 0, value);
3 sem_wait(&sem);    // decrement or block
4 /* critical section */
5 sem_post(&sem);    // increment
6 sem_destroy(&sem);
```

`sem_t sem` declares a semaphore object.

`sem_init(&sem, 0, value)` initializes semaphore with given arguments. The second argument specifies if the semaphore is shared only between threads of the same process (0) or could be shared across processes (1) (with shared memory). The third argument `value` is the initial semaphore count.

`sem_wait`

- If `value > 0` decreases `value` for 1 unit and thread continues.
- If `value == 0` thread blocks until another thread increases it.

`sem_post` increments the semaphore counter (`value`) and may wake a waiting thread.

Limiting Access to Resources (04_counting_sem_resource_pool.c)

```
1 #include <pthread.h>
2 #include <semaphore.h>
3 #include <stdio.h>
4 #include <unistd.h>
5
6 #define THREADS 5
7 #define RESOURCES 3
8
9 sem_t pool;
10 pthread_mutex_t print_lock = PTHREAD_MUTEX_INITIALIZER;
11
12 int active_threads = 0;
13
14 void* worker(void* arg) {
15     long id = (long)arg;
16
17     pthread_mutex_lock(&print_lock);
18     printf("T%d is waiting for a resource\n", id);
19     pthread_mutex_unlock(&print_lock);
20
21     sem_wait(&pool);    // acquire one of the 3 available resources
22
23     pthread_mutex_lock(&print_lock);
24     active_threads++;
25     printf("T%d acquired a resource | active = %d\n", id, active_threads);
26     pthread_mutex_unlock(&print_lock);
27
28     sleep(2);           // simulate using the resource
29
30     pthread_mutex_lock(&print_lock);
31     active_threads--;
32     printf("T%d released a resource | active = %d\n", id, active_threads);
```

```
33     pthread_mutex_unlock(&print_lock);
34
35     sem_post(&pool);    // release the resource
36
37     return NULL;
38 }
39
40 int main(void) {
41     pthread_t t[THREADS];
42
43     sem_init(&pool, 0, RESOURCES);
44
45     for (long i = 1; i <= THREADS; i++) {
46         pthread_create(&t[i - 1], NULL, worker, (void*)i);
47     }
48
49     for (int i = 0; i < THREADS; i++) {
50         pthread_join(t[i], NULL);
51     }
52
53     sem_destroy(&pool);
54     pthread_mutex_destroy(&print_lock);
55
56     return 0;
57 }
```

Example Output (order may vary)

```
T1 is waiting for a resource
T1 acquired a resource | active = 1
T2 is waiting for a resource
T2 acquired a resource | active = 2
T4 is waiting for a resource
T4 acquired a resource | active = 3
T3 is waiting for a resource
T5 is waiting for a resource
T1 released a resource | active = 2
T2 released a resource | active = 1
T3 acquired a resource | active = 2
T4 released a resource | active = 1
T5 acquired a resource | active = 2
T3 released a resource | active = 1
T5 released a resource | active = 0
```

At most 3 threads run inside the protected region (critical section with bounded concurrency) at the same time.

Other threads block at `sem_wait` until a resource is released.

The semaphore controls access to the shared resource pool.

A separate mutex prevents interleaved output, while the semaphore controls access to shared resources.

4.2 Special Case: Mutual Exclusion

If a semaphore is initialized to 1, it allows only one thread at a time and acts as a binary semaphore for mutual exclusion, similar to a mutex.

```
1 sem_t sem;
2 sem_init(&sem, 0, 1);
```

Practice Task

Change `RESOURCES` to 1 and observe the behavior.

5 Condition Variables

Mutexes and semaphores control access, but they do not allow threads to wait for specific conditions on shared data.

For example, a consumer should wait until data is available instead of repeatedly checking. Condition variables provide this mechanism.

5.1 Basic POSIX Usage

Waiting thread

```
1 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
2 pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
3
4 pthread_mutex_lock(&m);
5
6 while (condition_not_true)
7     pthread_cond_wait(&cond, &m);
8
9 /* condition is true */
10
11 pthread_mutex_unlock(&m);
```

Signaling thread

```
1 pthread_mutex_lock(&m);
2 /* update shared state */
3 pthread_cond_signal(&cond);
4 pthread_mutex_unlock(&m);
```

`pthread_cond_t cond` creates a condition variable object.

`pthread_cond_wait` releases the mutex and blocks until it is signaled.

When it returns (awakened by a signal), it re-acquires the mutex and returns from `pthread_cond_wait`.

We use `while (condition_not_true)` to re-check the condition after waking up, because spurious wakeups can happen, or another thread may consume the resource first.

`pthread_cond_signal(&cond)` wakes one waiting thread.

Waiting for Data (05_condvar_demo.c)

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 int ready = 0;
6 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
7 pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
8
9 void* consumer(void* arg) {
10     pthread_mutex_lock(&m);
11     printf("Consumer: checking condition\n");
12     while (ready == 0) {
13         printf("Consumer: waiting...\n");
14         pthread_cond_wait(&cond, &m);
15     }
16     printf("Consumer: condition met, proceeding\n");
17     pthread_mutex_unlock(&m);
18     return NULL;
19 }
20 void* producer(void* arg) {
21     sleep(1); // ensure consumer waits first
22     pthread_mutex_lock(&m);
23     printf("Producer: updating condition\n");
24     ready = 1;
25     printf("Producer: signaling condition\n");
26     pthread_cond_signal(&cond);
27     pthread_mutex_unlock(&m);
28     return NULL;
29 }
30 int main() {
31     pthread_t t1, t2;
32     pthread_create(&t1, NULL, consumer, NULL);
33     pthread_create(&t2, NULL, producer, NULL);
34     pthread_join(t1, NULL);
35     pthread_join(t2, NULL);
36     return 0;
37 }
```

Example Output

```
Consumer: checking condition
Consumer: waiting...
Producer: updating condition
Producer: signaling condition
Consumer: condition met, proceeding
```

The consumer does not use busy waiting.

It releases the mutex and blocks inside `pthread_cond_wait`, consuming no CPU.

After the producer signals, the consumer wakes up, re-acquires the mutex, and continues execution.

5.2 Broadcast and Multiple Threads

```
1 pthread_cond_broadcast(&cond);
```

Multiple threads may wake up at the same time.

Each thread must re-check the condition before proceeding.

6 Monitor

A monitor is a structured way to organize shared data together with:

- a mutex for mutual exclusion
- condition variables for waiting and signaling

In practice, a monitor is useful when threads must both protect shared data and wait for specific conditions on that data.

6.1 Example: Bounded Buffer Monitor

In this example, a producer inserts items into a bounded buffer and a consumer removes them.

The shared buffer state is protected by a mutex. Condition variables are used to block:

- the producer when the buffer is full
- the consumer when the buffer is empty

Code ([06_bounded_buffer_monitor.c](#))

```
1 #include <pthread.h>
2 #include <stdio.h>
3
```

```
4 #define N 5
5
6 int buffer[N];
7 int count = 0;
8
9 pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
10 pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
11 pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;
12
13 void put(int x) {
14     pthread_mutex_lock(&m);
15     while (count == N)
16         pthread_cond_wait(&not_full, &m);
17     buffer[count++] = x;
18     pthread_cond_signal(&not_empty);
19     pthread_mutex_unlock(&m);
20 }
21 int get() {
22     pthread_mutex_lock(&m);
23     while (count == 0)
24         pthread_cond_wait(&not_empty, &m);
25     int x = buffer[--count];
26     pthread_cond_signal(&not_full);
27     pthread_mutex_unlock(&m);
28     return x;
29 }
30 void* producer(void* arg) {
31     for (int i = 1; i <= 5; i++) {
32         put(i);
33         printf("Produced %d\n", i);
34     }
35     return NULL;
36 }
37 void* consumer(void* arg) {
38     for (int i = 1; i <= 5; i++) {
39         printf("Consumed %d\n", get());
40     }
41     return NULL;
42 }
43 int main() {
44     pthread_t p, c;
45     pthread_create(&p, NULL, producer, NULL);
46     pthread_create(&c, NULL, consumer, NULL);
47     pthread_join(p, NULL);
48     pthread_join(c, NULL);
49     return 0;
50 }
```

Possible Output (order may vary)

Produced 1 Produced 2

```
Produced 3
Produced 4
Produced 5
Consumed 1
Consumed 5
Consumed 4
Consumed 3
Consumed 2
```

The exact output order may vary because thread scheduling is nondeterministic.

6.2 POSIX Implementation Note

A monitor is not a new POSIX primitive; it is implemented using mutexes and condition variables.

6.3 How the Monitor Works

The producer calls `put()`. If the buffer is full, it waits on `not_full` until space becomes available.

The consumer calls `get()`. If the buffer is empty, it waits on `not_empty` until an item is produced.

After inserting an item, the producer signals `not_empty` to wake a waiting consumer.

After removing an item, the consumer signals `not_full` to wake a waiting producer.

6.4 Notes

This simplified buffer behaves as LIFO because it uses `buffer[-count]`. FIFO behavior would require separate input and output indices.

Practice Task (Debugging)

Replace `while` with `if` in both `put()` and `get()`.

Run the program multiple times.

What incorrect behavior can occur and why? Answer: Incorrect behavior may occur because the condition is checked only once. A thread can wake up even though the condition is no longer true. For example, multiple consumers may wake up, but the buffer may already be empty again when one of them continues. This can cause invalid accesses, incorrect values, or buffer underflow/overflow. Using `while` forces the thread to re-check the condition after waking up before proceeding.

7 Atomic Operations

Mutexes and semaphores use blocking to coordinate threads.

Another approach is to use **atomic operations**, which update shared data without locks.

7.1 Example: Atomic Counter

Code: `08_atomic_counter.c`

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdatomic.h>
4 #define THREADS 4
5 #define ITERATIONS 200000
6 atomic_int counter = 0;
7
8 void* worker(void* arg) {
9     for (int i = 0; i < ITERATIONS; i++) {
10         atomic_fetch_add(&counter, 1);
11     }
12     return NULL;
13 }
14 int main() {
15     pthread_t t[THREADS];
16     for (int i = 0; i < THREADS; i++)
17         pthread_create(&t[i], NULL, worker, NULL);
18     for (int i = 0; i < THREADS; i++)
19         pthread_join(t[i], NULL);
20     printf("Final counter: %d\n", counter);
21     return 0;
22 }
```

Program Output

Final counter: 800000

`atomic_fetch_add` performs the increment as a single atomic operation.

No updates are lost, even without using a mutex.

7.2 Key Idea

- Atomic operations avoid race conditions without locking.
- They are efficient for simple updates.
- They do not provide general coordination like mutexes or semaphores.